

Chapter 1: System Architecture and Technology Stack

1.1 Introduction

The OpenEvolve Dashboard represents an intersection of artificial intelligence, web technologies, and systems evaluation. Built with the MERN stack MongoDB, Express.js, React, and Node.js this platform serves as a dynamic web-based interface for analyzing the output data of OpenEvolve's program evaluation framework. Designed to be extensible and modular, the system provides users with the ability to visualize program metrics, interpret AI-driven evaluations, and interact directly with backing evaluation scripts in an integrated, browser-accessible interface.

The dashboard is deployed as a single-page application (SPA), which provides fast, smooth transitions among analysis views without unnecessary server reloads. The user can dive into in-depth data metrics, observe visual analytics, and call upon LLM-based reasoning to explain output results, creating a unified and interactive analytical environment.

1.2 MERN Stack

The decision to utilize the MERN stack is consistent with our emphasis on scalability, speed, and cross-platform compatibility. Each component plays a specific and complementary role:

- **MongoDB:** a NoSQL database that supports JSON-like document structures, ideal for flexible data storage such as metric results, evaluation logs, and user preferences. Although OpenEvolve can operate without persistent storage, MongoDB remains an optional enhancement to log metrics across sessions and enable comparative analysis between iterations.
- **Express.js:** the backend web application framework that connects Node.js with React. It facilitates secure API routes that retrieve processed data from the

OpenEvolve evaluator or other server-side scripts.

- **React:** the foundation for the front-end interface, providing the SPA experience. React components organize data visualization panels, user controls, and dynamically update metric outputs through hooks and context states.
- **Node.js:** the runtime environment that executes server-side JavaScript, hosting both the Express backend and the interaction layer with OpenEvolve's Python-based modules.

Together, these technologies create a robust ecosystem where the user interface, server logic, and AI evaluation modules communicate in real time.

1.3 Database Considerations

While MongoDB is fully integrated into the stack, its necessity is conditional. OpenEvolve's computational workflow, particularly its Python-based analysis scripts generates data that can be directly rendered on the front end without long-term storage. In scenarios where persistence is not required (e.g., single-run evaluation), JSON-based memory storage or temporary API state management suffices.

However, for research reproducibility and user-specific metrics tracking, MongoDB provides clear advantages:

- **Historical Comparison:** Stores past OpenEvolve runs for cross-metric analysis.
- **User Personalization:** Saves display settings and visualization filters.
- **Team Collaboration:** Enables shared access to previous evaluation results through user-specific collections.

This modular approach to database use allows the project to remain lightweight while still supporting scalability for larger research or production contexts.

1.4 Front-End Visualization and Metrics Dashboard

The OpenEvolve Dashboard emphasizes clarity, accessibility, and precision in visualizing computational data. Its metrics page central to the user experience aggregates the output data from OpenEvolve's metrics and presents it in visual forms. These data visualizations allow users to interpret performance, algorithmic efficiency, and other key measures in real time.

Each data update triggers a React state refresh, automatically redrawing the metrics display without requiring manual reloads. The goal is to present technical evaluation data in a format that is both intuitively readable and scientifically rigorous.



Initial Mockup (Figure 1)



Our Current Site (Figure 2)

1.5 Integration Between OpenEvolve and the Web Platform

The integration between OpenEvolve and the dashboard will be established through a secure API bridge. Node.js executes subprocess calls to Python, invoking `evaluator.py` and `init_program.py`. Once computation completes, the results are formatted into JSON and sent to the front end via Express routes.

React then consumes this data and displays it through its visualization components. This design enables high interoperability between the AI system's Python backend and the JavaScript-driven web client, creating a unified workflow for research analysis.

Chapter 2: Language Model Integration and Functional Workflow

2.1 Overview

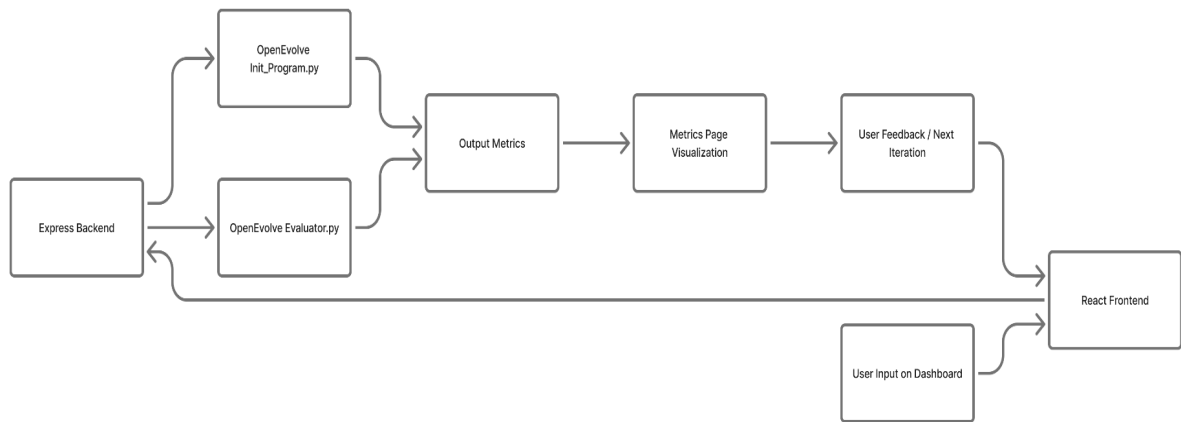
The integration of a large-language model (LLM) into the OpenEvolve Dashboard enhances the platform by offering the ability to produce `evaluator.py` and `init_program.py`. Natural-language explanations and contextual reasoning about the

system's metrics could also be offered to enhance its use. Specifically, the dashboard leverages ChatGPT , which serves as an on-demand research assistant. Rather than functioning as an open-ended chatbot, the model is strictly scoped to interpret and assist with two core modules of the system: evaluator.py and init_program.py.

2.2 Architecture of LLM Integration

The architecture for LLM integration resides within the MERN-stack framework defined in the Tech Stack chapter, linking user interactions through React (frontend) to Node.js/Express (backend), which in turn dispatches communication to the ChatGPT API. The workflow proceeds as follows:

1. **User Input** – The user submits a query via the “AI Assistant” panel on the dashboard.
2. **API Endpoint Handling** – An Express route receives the query and constructs a prompt for the ChatGPT API.
3. **Prompt Construction** – The backend composes a context string that includes relevant code fragments followed by the user's question, thus bounding the model's scope to only the domain-relevant content.
4. **Model Invocation** – The prompt is sent to the ChatGPT API and the model returns a response with the now start for evaluator.py and init_program.py.
5. **Response Filtering and Sanitization** – The backend examines the model's answer to ensure compliance with the domain constraints, eliminating any off-topic or irrelevant content.
6. **Frontend Delivery** – The sanitized explanation is returned to the React frontend and displayed in the “AI Assistant” panel for the user's review.



2.3 Data Handling, Privacy, and Context Management

Because OpenEvolve could deal with potentially sensitive data, the integration design prioritizes privacy, transparency, and minimal data exposure. Key safeguards include:

- Only non-sensitive code excerpts from evaluator.py and init_program.py are forwarded to the model. Full datasets, user-specific metrics, or raw logs are never sent to ChatGPT.
- The model's context window is strictly limited: only the relevant modules' code plus the user query, thereby reducing the risk of off-topic drift beyond the intended domain.
- No persistent storage of the model's responses is required unless specifically enabled for logging or auditing.
- The system maintains an audit trail of which code fragments were sent to the model and which user queries triggered them, ensuring reproducibility and transparency of the AI-assisted reasoning.

These practices align with recognized standards for responsible AI usage, ensuring that the LLM functions as a bounded assistant rather than an unfettered general-purpose tool.

2.4 Model Behavior, Scope, and Prompt Engineering

To ensure that the LLM remains focused and effective, we define specific behavioral roles and prompt engineering strategies:

- **Role Definition** – The system instructs the model explicitly: *“You are an assistant for the OpenEvolve Dashboard. You will only interpret the following code fragments and answer questions about how they operate.”*
- **Content Focus** – The model is restricted to tasks such as:
 - Explaining how a given function may be implemented and used within OpenEvolve
 - Describing how `init_program.py` sets up parameters, handles configurations, and orchestrates program execution.
 - Create the `evaluator.py` file and `init_program.py` file within the context of the problem
- **Prompt Structure** – The backend constructs the prompt with the following components:
 - System message specifying the assistant’s domain.
 - User Input: "I want to find $y = 3x + 5$ "

System sends to model:

"You are creating an evaluator and initial_program for a linear regression problem with formula $y = 3x + 5$.

Only output code relevant to this symbolic regression task."

- **Limitation Handling** – Because the free-tier model has finite context size and rate limits:
 - The system may truncate overly long modules and provide links or references instead.
 - If the model's output appears unsatisfactory, the system can retry with an adjusted prompt or notify the user that deeper analysis may require manual review.

Example Code: **add later**

2.5 Benefits and Limitations

Benefits include:

- The ChatGPT free model offers a cost-effective way to add interpretive AI to the dashboard without requiring custom-trained models or large infrastructure.
- By focusing strictly on `init_program.py` and `evaluator.py`, the assistant helps users understand the internal logic of OpenEvolve's evaluation pipeline, thus enhancing research transparency and user engagement.
- The integration encourages interactive learning—users can ask targeted questions and receive immediate responses, supporting both novice and

advanced users of the dashboard.

Limitations we may run into:

- The free-tier model is subject to rate limits, variable latency, and context-size constraints.
- The assistant cannot inspect live runtime data beyond what is provided in the prompt; it cannot modify behaviour or access private datasets.
- Because only code excerpts are included, the assistant may lack full context of the system's state or external dependencies, which can lead to incomplete or approximate answers.
- Over-reliance on the assistant may lead users to accept explanations uncritically; therefore, manual validation remains essential.